LECTURE 13

Templates & Exceptions

Generic programming and runtime error handling in C++

What we will cover today

01 The Need for Templates

Why function overloading isn't enough, and how generics solve it.

02 Function Templates

Syntax, type parameters, and worked examples.

03 Class Templates

Generic classes, Stack<T>, Pair<T,U>, instantiation rules.

04 Templates in UML

Parameterized class notation, dependencies, and stereotype.

05 Exceptions in C++

try / throw / catch, parametrized exceptions, multiple handlers.

06 Practice

MCQs and short coding problems to lock the concepts in.

Introduction

What is a template?

Templates make it possible to use ONE function or class to handle many different data types.

WHY?

- **Reuse**
- **Type safety**
- **Performance**

TWO TYPES

- **Function templates**
- **Class templates**

The problem — function overloading

Before templates, we wrote one overload per type. Same body, different signatures.

```
int abs(int n) {
    return (n < 0) ? -n : n;
}

long abs(long n) {
    return (n < 0) ? -n : n;
}

float abs(float n) {
    return (n < 0) ? -n : n;
}
```

Issues with function overloading

1. **Repetition**
2. **Wasted space**
3. **Error multiplication**
4. **Closed to new types**

Function template — the idea

Write the algorithm once, parameterised on a type T.

- `template<typename T>`
- T is a placeholder. When you call the function with an int, T becomes int. When you call it with a string, T becomes string.

```
#include<iostream>
Using namespace std;
template <typename T>
T myFunc(T a, T b) {
   return a + b; }
int main(){
myFunc(2, 3);
myFunc(1.5, 2.5);}
```

Function template — the idea

Write the algorithm once, parameterised on a type T.

- The compiler generates a separate, type-correct version for each type. This is called template instantiation.
- You can use **`class`** or **`typename`** interchangeably for the parameter keyword.
- Multiple parameters are allowed: **`template<typename T, typename U>`**.

```
#include<iostream>
Using namespace std;
template <typename T>
T myFunc(T a, T b) {
   return a + b; }
int main(){
myFunc(2, 3);
myFunc(1.5, 2.5);}
```

Function template

```
template <typename T>
T myFunc(T arg1, T arg2)
{   /* body uses T like a normal type */ }
```

template<...>	typename T	T myFunc	T arg1, T arg2
Tells the compiler that what follows is a template, not a regular function.	Declares T as a generic type parameter. 'class T' is equivalent.	T is the return type; it could also have been a fixed type like bool or void.	Parameters whose type is determined when the function is called.

Example

```
template <typename T>
T square(T x) {
    return x * x;
}
int main() {
    cout << square(5)   << endl;
    cout << square(2.5) << endl;}
```

```
OUTPUT
25
6.25
```

How it works

- `square(5)` compiler sees an int argument, generates int square(int).
- `square(2.5)` compiler generates double square(double).
- Both versions exist in the final binary; the source had only one.

Example — add() with two type parameters

```
template <typename T, typename U>
auto add(T a, U b) {
    return a + b;}

int main() {
    cout << add(3, 4.5) << endl;
    cout << add(2, 7)   << endl;
}
```

```
OUTPUT
7.5
9
```

Notice

- Two type parameters: `T` and `U` can differ.
- `auto` lets the compiler figure out the return type from a + b.
- `add(3, 4.5)` int + double promotes to double, result is 7.5

Example — scale() with a non-template parameter

```
template <typename T>
T scale(T value, int factor) {
    return value * factor;}
int main() {
    cout << scale(3,   4) << endl;
   cout << scale(2.5, 3) << endl;
}
```

```
OUTPUT
12
7.5
```

Key insight

- Not every parameter has to be templated.
- **`value`** is generic (T) but **`factor`** is always int.

Syntax variation

You may see legacy code that uses class instead of typename.

```
template <class atype>
int find(atype* array, atype value, int size)
{
    /* function body */
}
```

class vs typename

- Identical for type parameters.
- `class` is the older keyword; `typename` reads better when T is not actually a class.
- Modern style prefers typename.

Naming the type parameter

- Any identifier is allowed: `T`, `U`, `atype`, `ItemType`.
- Convention: short single letters (T, U, V) for general code; descriptive names for libraries.

Class templates

Same idea, but the WHOLE class is parameterized on a type.

- A class template is a blueprint: instantiate it with a concrete type to get a real class.
- **`Stack<int>`** and **`Stack<string>`** are completely separate types. They share no objects.
- When you define a member function outside the class, you must repeat the template header.

```
template <typename T>
class Box {
  T value;
public:
  Box(T v) : value(v){}
  T get() const {
      return value; }};
Box<int> a(7);
Box<string> b("hi");
```

Class template example — Stack<T>

```
template <typename T>
class Stack {
    static const int MAX = 100;
    T data[MAX];
    int top = -1;
public:
    void push(T x) {
        if (top>=MAX-1) throw "overflow";
        data[++top] = x;}
    T pop() {
        if (top < 0) throw "underflow";
        return data[top--];}
    bool empty() const { return top < 0;}
};
```

```
int main() {
    Stack<int> s;
    s.push(10);
    s.push(20);
    cout << s.pop();
    cout << s.pop();
    Stack<string> t;
    t.push("hi");
    cout << t.pop();}
```

Class template example — Pair<T, U>

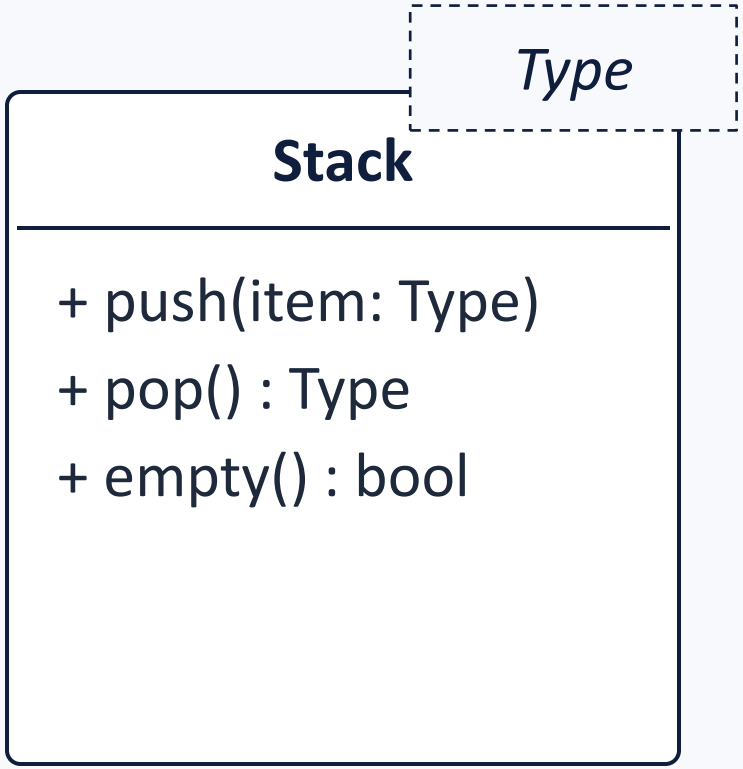
```
template <typename T, typename U>
class Pair {
    T first;
    U second;
public:
    Pair(T a, U b) : first(a), second(b) {}

    T getFirst() const {return first;  }
    U getSecond() const { return second; }

    void print() const {
        cout << "(" << first << ", " << second << ")";
    }};
```

Class template example — Pair<T, U>

```
int main() {
    Pair<string,int> p1("age", 21);
    Pair<int,double>  p2(3, 1.5);
    p1.print();
    p2.print();}
```

Templates in UML

Templates (parameterized classes) get a special class symbol with a dotted rectangle holding the template parameters.

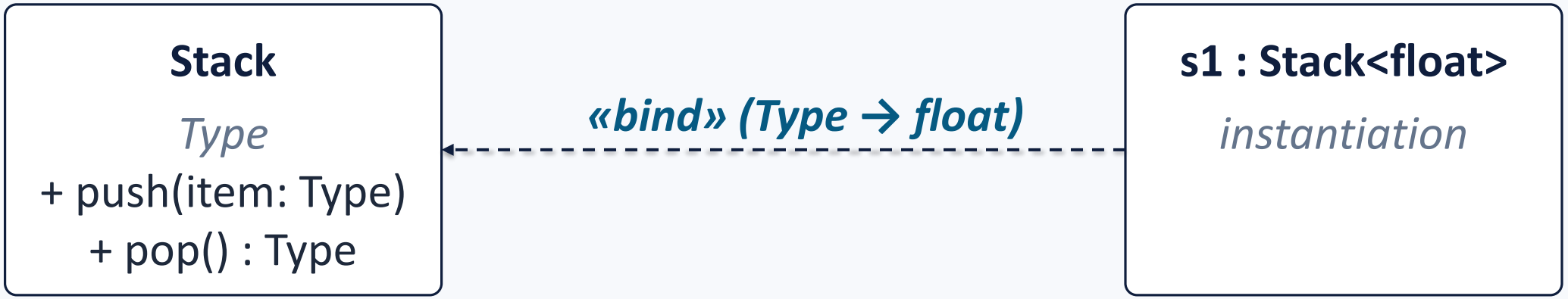


Reading the diagram

- Dotted rectangle in the upper-right holds template parameters (Type).
- Operations show return types after a colon (UML convention), e.g. **pop() : Type**.
- Type appears wherever it is used in signatures.
- Specific instantiations (e.g. **s1: Stack<float>**) are drawn as separate classes that depend on the template.

UML — dependencies & stereotypes

- **A dependency** is a "uses" relationship: a change in the independent element may force a change in the dependent one.
- The template class is the independent element; classes instantiated from it are dependent.
- Drawn as dashed arrow pointing from dependent element to the independent one.
- **Stereotypes** (in «guillemets») label what kind of relationship it is.
- **«bind»** means "instantiate the template by binding parameters", e.g. **«bind» (Type → float)**.



Introduction to Exceptions

What is an exception?

A systematic, object-oriented way to handle ERRORS that occur at runtime. The program can not foresee them at compile time.

Out of memory	File not found	Bad value
new fails because the heap is exhausted.	open() returns failure for a missing path.	Constructor receives an impossible argument (e.g. negative age).

Bad index	Divide by zero	Network error
vector<T> accessed with at(i) where i is out of range.	Arithmetic that the CPU/program defines as undefined.	Socket disconnects mid-read.

Why use exceptions?

Two ways to signal an error: return a special value, or throw.

Error codes (old way)

```
int r = openFile("x");
if (r == -1) {/*handle
error here*/}
else {/* normal flow */}
```

- Mixes "good" and "error" logic in the same function.
- Hard to propagate up multiple call levels.

Exceptions (modern way)

```
try {
    openFile("x");
    /* normal flow */}
catch (FileError& e) {
    /* handle error here*/}
```

- Cannot be silently ignored, uncaught throws terminate the program.
- Normal logic stays clean; error logic lives in catch blocks.

Sequence of events

Five steps from normal execution to handled error.

1	2	3	4	5
Normal	**Enter try**	**Error occurs**	**throw**	**Catch**
Code is executing normally outside any try block.	Control enters a try block.	A statement causes an error in a member function.	The function throws an exception object.	Control transfers to the matching catch block following the try.

When an exception is thrown, the rest of the try block is SKIPPED and execution jumps straight to the matching catch handler. Functions on the stack are unwound, and local objects are destroyed in reverse order of construction.

Worked example — divide by zero

```
#include <iostream>
using namespace std;

double divide(double a, double b) {
    if (b == 0)
        throw "Division by zero!";
    return a / b;
}

int main() {
    try {
        cout << divide(10, 2) << endl;
        cout << divide(7, 0)  << endl;
        cout << "never printed\n";
    }
    catch (const char* msg) {
        cout << "Caught: " << msg << endl;
    }
}
```

OUTPUT

```
5
Caught: Division by zero!
```

Trace

- **divide(10,2)** returns 5 normally.
- **divide(7,0)** throws — the rest of try is skipped.
- catch matches const char*, prints message.
- Program continues after the catch block.

Multiple catch blocks

A try block can have several handlers and the first matching type wins.

```
try {
    process(input);
}
catch (int code)
{ cout << "int   : " << code; }
catch (const char* msg)
{cout<<"cstr  : " << msg;  }
catch (std::exception& e)
{cout<<"std::ex:"<< e.what(); }
catch (...){ cout << "unknown error";   }
```

Rules to remember

- Catches are tested top-to-bottom.
- Order from most specific (derived) to most general (base).
- **catch (...)** matches anything so put it last.
- Catching by reference (**std::exception&**) avoids slicing.
- If no handler matches, std::terminate is called.

Parametrized exceptions

Often you want the thrown object to CARRY data: which value, which field, which line.

```
class AgeError {
public:
    int badValue;
    string field;
    AgeError(int v, string f) : badValue(v),
field(f) {}
};
void setAge(int age) {
    if (age < 0 || age > 150)
        throw AgeError(age, "age");}
int main() {
    try { setAge(-5); }
    catch (AgeError& e) {
        cout << "Bad " << e.field
             << " = " << e.badValue << endl;}}
```

OUTPUT

```
Bad age = -5
```

Why this matters

- The exception object stores context.
- The handler can show a meaningful message, log to a file, retry, etc.

try / throw / catch — minimal example

```
class AClass {
public:
    class AnError { };
    void Func() {
        if (/* error condition */)
            throw AnError();}};

int main() {
    try {
        AClass obj1;
        obj1.Func();
    }
    catch (AClass::AnError) {   // catch block
        cout << "oops\n";
    }
    return 0;}
```

Three keywords

try { … }

- wraps code that may throw.

throw expr;

- raises an exception. expr can be any object.

catch (Type x) { … }

- handles a thrown exception of matching type.
- **AnError** here is just a tag with no data, but its type alone identifies the error.

Standard library exceptions

C++ ships a hierarchy of ready-made exception classes in <stdexcept>.

```
#include <stdexcept>
#include <iostream>
using namespace std;
int factorial(int n) {
    if (n < 0)
      throw invalid_argument("n must be >= 0");
    int r = 1;
    for (int i = 2; i <= n; ++i) r *= i;
    return r;
}
int main() {
    try { cout << factorial(-3); }
    catch (const exception& e) {
        cout << "Error: " << e.what();}}
```

`std::exception`

Base class for all standard exceptions. Has virtual what().

`std::logic_error`

Programmer mistake (e.g. invalid_argument, out_of_range).

`std::runtime_error`

Failure that could only be detected at runtime (e.g.

`std::bad_alloc`

Thrown by new when memory allocation fails.

Best Practices

DO Throw by value, catch by reference.

DO Derive your own exceptions from std::exception.

DO Order catch blocks specific → general.

DON'T Use exceptions for normal control flow.

DON'T Throw raw pointers (throw new T(...)).

DON'T Let exceptions escape destructors.

Practice — MCQs (1 of 2)

Q1. template<typename T> means…

A. T is always int

B. T is a placeholder for any type, fixed at instantiation

C. T can only be a class, not a primitive

D. T is a runtime variable

Q2. Which keyword raises an exception?

A. catch

B. raise

C. throw

D. panic

Q3. What happens if no catch matches a thrown exception?

A. The throw is silently ignored

B. std::terminate is called and the program ends

C. It is converted to a return -1

D. The compiler reports an error

Practice — MCQs (2 of 2)

Q4. Stack<int> and Stack<string> are…

A. the same type

B. different but interconvertible

C. two completely separate types

D. illegal

Q5. Catching by reference (catch (Err& e)) instead of by value avoids…

A. multiple inheritance

B. object slicing and an extra copy

C. template instantiation

D. compile errors

Q6. Which is the BEST base class for your own exception types?

A. void

B. std::exception

C. std::string

D. int

MCQ answer key

Q1	**B**	T is a placeholder, replaced with the actual type when the template is instantiated.
Q2	**C**	throw raises an exception; catch handles it; try wraps protected code.
Q3	**B**	An uncaught exception terminates the program via std::terminate.
Q4	**C**	Each instantiation generates a distinct, unrelated class type.
Q5	**B**	Catching by reference avoids slicing and the cost of a copy.
Q6	**B**	std::exception is the standard base because what() integrates with std code.

Practice

CQ1 maximum() function template

Write a function template max3 that returns the largest of three values of the same type T. Test it with int, double, and string.

CQ2 Class template Queue<T>

Implement a simple FIFO Queue<T> with enqueue(T), T dequeue(), and bool empty(). Throw an exception on dequeue from an empty queue.

CQ3 Parametrized exception

Create a class BankError that stores an error message and an account ID. Throw it from withdraw(amount) when the balance is insufficient, and catch it in main().

CQ4 Exception-safe input

Write a function readPositiveInt() that throws invalid_argument when the user types a negative number or a non-number, and keeps prompting until valid input is given.

RECAP

Templates & Exceptions

Templates
Write code once, generate type-correct versions for many types.

Function templates
Type-parameterized functions; the compiler picks T from the call.

Class templates
Type-parameterized classes Stack<T>, Pair<T,U>

Exceptions
Object-oriented runtime error handling

Parametrized exceptions
Exception objects carry context

Best practice
Throw by value, catch by reference; specific catches first.